

Section-6 Task Report

Overview:

This task covers building connected modules, using different coding styles, and setting up automatic tests in SystemVerilog.

- **The Design:** built a Half Adder using two different coding methods (assign and always_comb). Then, wired these Half Adders together to create a Full Adder.
- **Automated Verification:** A self-checking testbench used for loops to test every possible input combination (4 states for the Half Adders and 8 states for the Full Adder).
- **Dynamic Checking:** The testbench automatically compared the two Half Adders to ensure they matched exactly. It also checked the Full Adder's output against the correct mathematical answer.
- **Results:** All tests passed successfully. The simulation generated a clean waveform and terminal logs showing 100% accurate results.

EDA Playground Link: <https://www.edaplayground.com/x/KEac>

design.sv:

```
`timescale 1ns / 1ps

module half_adder1(a, b, sum, carry);
input logic a, b;
output logic sum, carry;

assign sum = a ^ b;
assign carry = a & b;

endmodule

module half_adder2(a, b, sum, carry);
input logic a, b;
output logic sum, carry;

always_comb begin
case({a,b})
    2'b00: begin
        sum = 0;
        carry = 0;
    end
    2'b01: begin
        sum = 1;
        carry = 0;
    end
    2'b10: begin
```

```

        sum = 1;
        carry = 0;
    end
    2'b11: begin
        sum = 0;
        carry = 1;
    end

    default: begin
        sum = 0;
        carry = 0;
    end
endcase
end
endmodule

module full_adder(a, b, sum, cin, cout);
input logic a, b, cin;
output logic sum, cout;

logic sum1, carry1, carry2;

half_adder1 halfadder1(.a(a), .b(b), .carry(carry1), .sum(sum1));
half_adder2 halfadder2(.a(sum1), .b(cin), .sum(sum), .carry(carry2));

assign cout = carry1 | carry2;

endmodule

```

testbench.sv:

```

`timescale 1ns / 1ps

module adder_tb();
logic a, b, cin, cout, sum, sum1, carry1, sum2, carry2;

int i;

half_adder1 uut_ha1 (.a(a), .b(b), .sum(sum1), .carry(carry1));
half_adder2 uut_ha2 (.a(a), .b(b), .sum(sum2), .carry(carry2));
full_adder uut_fa (.a(a), .b(b), .cin(cin), .cout(cout), .sum(sum));

task automatic check (input logic [1:0] actual, expected, string name);
    if(actual !== expected)
        $display("[FAIL] %s: Actual = %02b , Expected = %02b at Time = %0t", name, actual, expected, $time);
    else

```

```

        $display("[PASS] %s: Value = %02b at Time = %0t", name, actual,
$time);
    endtask

function automatic logic [1:0] exp_fa(input logic a,b,cin);
    return a + b + cin;
endfunction

initial begin

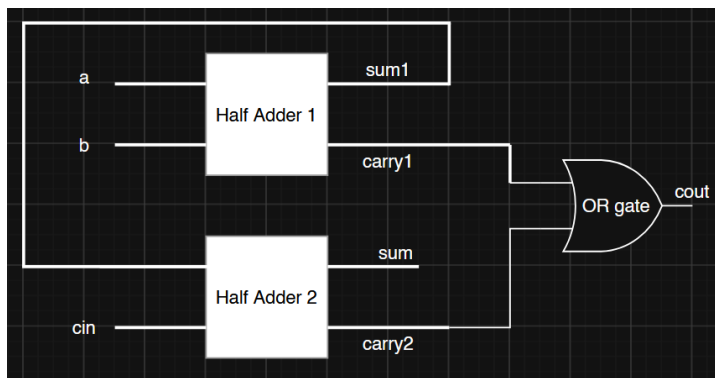
    $dumpfile("dump.vcd");
    $dumpvars(0, adder_tb);

    $display("--- Starting Half Adder Tests ---");
    for(i = 0; i < 4; i++) begin
        {a, b} = i; #5;
        check({carry2, sum2}, {carry1, sum1}, "Half Adder");
    end

    $display("--- Starting Full Adder Tests ---");
    for(i=0; i<8; i++) begin
        {a,b,cin} = i; #5;
        check({cout, sum}, exp_fa(a,b,cin), "Full Adder");
    end
$finish;
end
endmodule

```

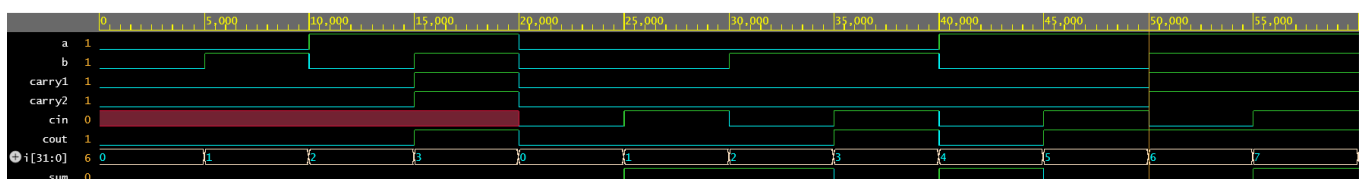
Schematic diagram:



Terminal Output:

```
--- Starting Half Adder Tests ---
[PASS] Half Adder: Value = 00 at Time = 5000
[PASS] Half Adder: Value = 01 at Time = 10000
[PASS] Half Adder: Value = 01 at Time = 15000
[PASS] Half Adder: Value = 10 at Time = 20000
--- Starting Full Adder Tests ---
[PASS] Full Adder: Value = 00 at Time = 25000
[PASS] Full Adder: Value = 01 at Time = 30000
[PASS] Full Adder: Value = 01 at Time = 35000
[PASS] Full Adder: Value = 10 at Time = 40000
[PASS] Full Adder: Value = 01 at Time = 45000
[PASS] Full Adder: Value = 10 at Time = 50000
[PASS] Full Adder: Value = 10 at Time = 55000
[PASS] Full Adder: Value = 11 at Time = 60000
$finish called from file "testbench.sv", line 40.
$finish at simulation time          60000
      V C S   S i m u l a t i o n   R e p o r t
```

Waveform:



Synthesis:

assign and always_comb synthesize to the same gates because the compiler looks at the underlying logic, not the code syntax.

- **assign** gives the compiler the explicit math equations.
- **always_comb** gives the compiler a truth table.

Because both methods describe the exact same input-to-output relationship, the synthesis tool realizes they are functionally identical and optimizes them into the exact same physical hardware circuit.